

Unde lucrezi: `vanilla-ts/src/exercitii/`, în fișierele scrise de tine la 01–06. Fiecare exercițiu spune fișierul. Clasa nouă o scrii sub clasele existente; codul din **în main** îl adaugi la finalul fișierului, sub codul de rulare existent, care rămâne neatins.

Nou, ai voie: `extends`, `super(...)` în constructor, `super.metoda()`, `protected`.

Încă NU folosim: `abstract` nou (ce e deja abstract din 06 rămâne), `interface`, `instanceof`, `static`, `any`.

Cum se citește fișa: rândul **Teorie** îl citești *înainte* de exercițiu (demo-ul din `src/teorie/`, rulat). Caseta **Indiciu** o citești *după* prima încercare. **Ieșirea așteptată** trebuie să iasă literal; verifici cu `npx tsx` și cu `npx tsc --noEmit` fără erori.

Partea 1 — adaugi un tip nou într-o ierarhie

1. Raportul cu dată

nivel 1

Fișierul `03-raport/main.ts`. Scrie o clasă nouă:

- `class RaportCuData extends Raport`.
- Câmp `readonly data: string`.
- `constructor(titlu: string, data: string)`: cheamă `super(titlu)`, apoi pune `data` în câmp.
- `text(): string` întoarce linia `Generat: <data>`, apoi tot ce întoarce `text()` din `Raport`. Nu rescrii titlul și liniile: le iei gata compuse prin `super.text()`.

În **main**:

```
const cuData = new RaportCuData("Rezultate suita", "2026-10-02");
cuData.adauga("3 teste verzi");
cuData.adauga("1 test rosu");
console.log(cuData.text());
```

Ieșire așteptată (sub ieșirea veche a lui `RaportSemnat`, care rămâne identică):

```
Generat: 2026-10-02
Rezultate suita
3 teste verzi
1 test rosu
```

Teorie: `B1-mostenire-extends-super` — `InregistrareEroare: super(moment, text)` în constructor și `super.formateaza()` în metoda rescrisă.

Indiciu: `super(titlu)` e obligatoriu prima linie din constructor. În `text()`, tot ce ai de făcut e `return `Generat: ${this.data}\n${super.text()}``; — dacă te trezești că parcurgi liniile, ai luat-o pe drumul greșit.

Gata când: ies cele patru linii de mai sus și ieșirea veche a fișierului e neschimbată.

De reținut: subclasa adaugă, nu rescrie. `super.text()` e locul în care refolosești în loc să copiezi.

2. Numărătorul cu pas

nivel 1

Fișierul `04-numarator/main.ts`. Scrie:

- `class NumaratorCuPas extends Numarator`.
- Câmp `readonly pas: number`; `constructor(pas: number)` cu `super()` pe prima linie.
- `urmatorul(): number` crește valoarea cu `pas` și întoarce noua valoare.

NumaratorCuLimita rămâne exact cum e.

În `main`:

```
const cuPas = new NumaratorCuPas(5);
const valori = [cuPas.urmatorul(), cuPas.urmatorul(), cuPas.urmatorul()];
console.log(valori.join(" "));
```

leșire așteptată (a treia linie a fișierului):

```
5 10 15
```

Teorie: `B2-mostenire-protected` — subclasa scrie direct în câmpul `protected` al bazei.

Indiciu: la 04 ai pus `valoare` `protected`, deci aici scrii `this.valoare += this.pas;`. Nu chema `super.urmatorul()`: ar mai adăuga un 1 pe deasupra.

Nota: apare `5 10 15`, iar liniile `1 2 3` și `1 2 2` sunt identice cu cele de dinainte.

De reținut: două subclase ale aceleiași baze nu știu una de alta. Ai adăugat un tip fără să atingi nici baza, nici sora lui.

Partea 2 — subclasa pune o regulă peste bază

3. Cheia nu se repetă

mai greu

Fișierul `01-set-de-date/main.ts`. Scrie:

- `class SetDeDateCuCheie extends SetDeDate`.
- Câmp `readonly cheie: string`; constructor(`coloane: string[], cheie: string`) cu `super(coloane)`.
- `adauga(rand: string[]): void`, în trei pași: (a) ia din `rand` valoarea de pe poziția coloanei-cheie; (b) dacă valoarea e deja printre `this.valorile(this.cheie)`, aruncă `new Error(`Duplicate key ${valoare}`)`; (c) altfel cheamă `super.adauga(rand)`.

În `main`:

```
const cuCheie = new SetDeDateCuCheie(["ESID", "FR_GUID", "STATUS"], "ESID");
cuCheie.adauga(["ESID-100", "FR-1389", "Active"]);
cuCheie.adauga(["ESID-777", "FR-B777", "Pending"]);
try {
  cuCheie.adauga(["ESID-100", "FR-X", "Blocked"]);
} catch (e) {
  console.log((e as Error).message);
}
try {
  cuCheie.adauga(["ESID-778", "FR-B778"]);
} catch (e) {
  console.log((e as Error).message);
}
console.log("randuri:", cuCheie.numarRanduri());
```

leșire așteptată:

```
Duplicate key ESID-100
Row has 2 values, expected 3
randuri: 2
```

Teorie: `B2-mostenire-protected` — `ListaFaraDuplicate.adauga` verifică, apoi cheamă `super.adauga(rand)`.

Indiciu: poziția coloanei-cheie e `this.coloane.indexOf(this.cheie)`. Asta merge doar dacă `coloane` nu e `private` în `SetDeDate`: dacă la 01 l-ai pus `private`, schimbă-l în `protected` (subclasa îl vede, main tot nu). Pasul (c) e obligatoriu, altfel rândul bun nu se mai adaugă nicăieri și regula „3 valori” dispare.

Gata când: ies cele trei linii de mai sus; a doua dovedește că regula bazei a rămas în picioare.

De reținut: un override care nu cheamă `super.adauga(...)` pierde regulile bazei, în tăcere.

4. Nicio valoare goală

mai greu

Fișierul `01-set-de-date/main.ts`. Scrie:

- `class SetDeDateStrict extends SetDeDate`, fără constructor propriu (îl moștenește pe al bazei).
- `adauga(rand: string[]): void`: parcurge `rand` cu un index `i`; dacă `rand[i].trim() === ""`, aruncă `new Error(`Empty value in column ${this.coloane[i]}`)`. La final cheamă `super.adauga(rand)`.

În `main`:

```
const strict = new SetDeDateStrict(["ESID", "FR_GUID", "STATUS"]);
strict.adauga(["ESID-100", "FR-1389", "Active"]);
try {
  strict.adauga(["ESID-777", " ", "Pending"]);
} catch (e) {
  console.log((e as Error).message);
}
try {
  strict.adauga(["ESID-779", ""]);
} catch (e) {
  console.log((e as Error).message);
}
console.log("randuri:", strict.numarRanduri());
```

Ieșire așteptată dacă verificarea ta stă *înainte* de `super.adauga`:

```
Empty value in column FR_GUID
Empty value in column FR_GUID
randuri: 1
```

Mută apelul `super.adauga(rand)` pe prima linie a metodei și rulează din nou: a doua linie devine `Row has 2 values, expected 3`. Alege ordinea pe care o consideri corectă și scrie într-un rând de ce.

Teorie: `B2-mostenire-protected`.

Indiciu: rândul `["ESID-779", ""]` încalcă *ambele* reguli. Care mesaj iese depinde doar de ce verificare rulează prima — a ta sau a bazei. `super.adauga(rand)` e un apel obișnuit: poate sta oriunde în metodă.

Gata când: ieșirea se potrivește cu ordinea aleasă de tine, iar `randuri: 1` arată că niciun rând refuzat n-a intrat.

De reținut: ordinea apelului la `super` se vede în rezultat. Nu e detaliu de stil.

Partea 3 — provocări

5. Numărătorul ciclic, al treilea etaj

provocare

Fișierul `04-numarator/main.ts`. Scrie:

- `class NumaratorCiclic extends NumaratorCuLimita`, fără constructor propriu.
- `urmatorul(): number`: dacă valoarea a ajuns la limită, pune valoarea înapoi la 0; apoi întoarce `super.urmatorul()`.

Constrângere: nu modifici `Numarator` și `NumaratorCuLimita` (cu o excepție: dacă la 04 ai pus `limita` `private`, fă-o `protected`).

În `main`:

```
const ciclic = new NumaratorCiclic(3);
const valori: number[] = [];
for (let i = 0; i < 7; i++) {
    valori.push(ciclic.urmatatorul());
}
console.log(valori.join(" "));
```

leșire așteptată:

1 2 3 1 2 3 1

Teorie: `B1` + `B2`: lanțul `super` pe două niveluri.

Indiciu: după ce ai pus valoarea pe 0, `super.urmatatorul()` (cel din `NumaratorCuLimita`) vede că nu e la limită și cheamă la rândul lui `Numarator`, care face `0 → 1`. Dacă ai scris `this.valoare = 1; return 1;`, ai rescris logica în loc s-o folosești: funcționează, dar nu e exercițiul.

Gata când: ies cele șapte numere, iar în `NumaratorCiclic` nu apare nici `++`, nici `+ 1`.

De reținut: fiecare etaj adaugă o regulă și o lasă pe a celorlalți în picioare, prin `super`.

6. Raportul compus, fără moștenire

provocare

Fișierul `03-raport/main.ts`. Scrie:

1. `class RaportCompus` — **fără** `extends`.
2. Două câmpuri `readonly` de tip `Raport`: `primul` și `alDoilea`, date prin constructor.
3. `text(): string` = textul primului, o linie `---`, textul celui de-al doilea.

În `main`: `new RaportCompus(raport, cuData)`, unde `raport` e `RaportSemnat`-ul de la 03 și `cuData` obiectul de la exercițiul 1; afișează `text()`.

leșire așteptată:

```
Rezultate suita
3 teste verzi
1 test rosu
-- andreea
---
Generat: 2026-10-02
Rezultate suita
3 teste verzi
1 test rosu
```

Scrie în două rânduri: dacă `RaportCompus` ar fi `extends Raport`, ar moșteni `adauga(linie)`. În care dintre cele două rapoarte ar intra linia?

Teorie: `B1` — ultimele două linii din `main`: un obiect al subclasei stă într-o variabilă de tipul bazei.

Indiciu: câmpurile sunt de tip `Raport`, deci primesc orice subclasă. Dacă nu găsești un răspuns bun la întrebarea cu `adauga`, ai găsit motivul: moștenirea ți-ar da o metodă fără sens.

Gata când: ies cele nouă linii de mai sus și în clasa nouă nu apare cuvântul `extends`.

De reținut: moștenirea spune „este un”, compoziția spune „are un”. Când iei moștenire doar ca să refolosești o metodă, plătești cu metode pe care nu le vrei.

7. Clasa din mijloc, pentru exportatoarele cu separator

provocare

Fișierul `06-exportatoare/main.ts`. `ExportatorCsv` și `ExportatorTabelText` au același `converteste`, cu alt separator. Scrie:

1. `class ExportatorCuSeparator` extends `Exportator`, cu `readonly extensie: string`, `protected readonly separator: string`, constructor(`extensie: string`, `separator: string`) (cu `super()` prima linie) și `converteste` scris o singură dată, cu `this.separator`.
2. `ExportatorCsv` devine `extends ExportatorCuSeparator` și conține **doar** constructor() { `super("csv", ",");` }. La fel `ExportatorTabelText` cu `" | "`.
3. `ExportatorJson` rămâne neschimbat.
4. Adaugă `class ExportatorTsv` extends `ExportatorCuSeparator` cu `super("tsv", "\\t")` și pune-l la finalul array-ului.

leșire așteptată: cea de la 06, identică, plus la final:

```
raport.tsv
ESID-100      Active
ESID-777      Pending
```

Scrie într-un rând de ce `ExportatorJson` nu încapă în clasa din mijloc.

Teorie: B1 (constructor cu `super`) și C2 (`extensie` e abstractă în `Exportator`; clasa din mijloc o face concretă).

Indiciu: `super()` e obligatoriu în constructorul oricărei clase cu `extends`, chiar dacă baza n-are constructor scris. Corpul lui `converteste` e cel pe care îl ai deja în `Csv`, cu `","` înlocuit de `this.separator`. JSON nu unește valori cu un separator: dă tot array-ul lui `JSON.stringify`.

Gata când: `converteste` apare în exact două locuri (clasa din mijloc și JSON), iar `ExportatorTsv` are cel mult patru linii.

De reținut: clasa din mijloc ține ce au în comun *doar unele* subclase. Nu urci în bază ce nu e al tuturor.